

```
{ticks.map((tick, i) => (
  <text
    key={tick}
    className="Axis__tick"
    transform={`translate(${scale(tick)}, 25)`}
  >
    { formatTick(tick) }
  </text>
))}
```

When we look at our chart, we can see a wonderful x axis with ticks:



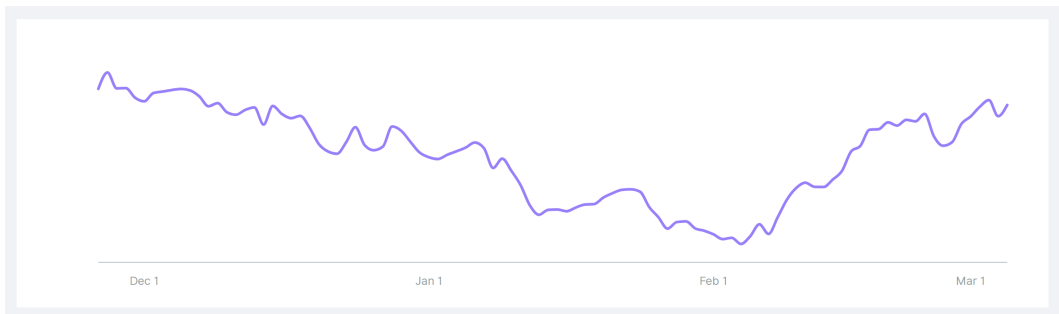
timeline with x axis

Those dates don't look right, though. Note that our react **Axis** component accepts a `formatTick` prop, which will be a function that takes a tick and converts it into a human-readable string. d3 axis generators have built-in logic that will detect date strings and format them correctly.

Let's override the default `formatTick` prop and pass `formatDate` defined at the top of `src/Timeline.jsx`.

```
<Axis
  dimension="x"
  scale={xScale}
  formatTick={formatDate}
/>
```

Much better!



timeline with x axis

In your own React chart library, it might be a good idea to detect whether or not the tick is a date object and format it accordingly. That will depend on your use cases: how often will you need to format dates? Will you want all dates to be formatted the same way?

Lastly, we'll want to render the label for our axis. Since we might not always want an axis label, we'll check if the `label` prop exists before rendering our label. We'll also horizontally center our label and shift it down 60 pixels to give our ticks space.

```
{label && (
  <text
    className="Axis__label"
    transform={`translate(${dimensions.boundedWidth / 2}, 60)`}
  >
    { label }
  </text>
)}
```

Our `AxisVertical` will look very similar to `AxisHorizontal`. We'll start with the same basic component.

```

function AxisVertical (
  { dimensions, label, formatTick, scale, ...props }
) {
  return (
    <g className="Axis AxisVertical" {...props}>
      </g>
    )
  }
}

```

Try to fill out the component as much as possible without looking about the completed code below. You might want to tweak the `numberOfTicks` parameter — these ticks will be stacked vertically and might have more room.

Remember that your y axis label will need to be rotated -90 degrees to fit.

```

function AxisVertical (
  { dimensions, label, formatTick, scale, ...props }
) {
  const numberOfTicks = dimensions.boundedHeight / 70

  const ticks = scale.ticks(numberOfTicks)

  return (
    <g className="Axis AxisVertical" {...props}>
      <line
        className="Axis__line"
        y2={dimensions.boundedHeight}
      />

      {ticks.map((tick, i) => (
        <text
          key={tick}
          className="Axis__tick"
          transform={`translate(-16, ${scale(tick)})`}
        >
          { formatTick(tick) }

```

```

    </text>
  )})

  {label && (
    <text
      className="Axis__label"
      style={{
        transform: `translate(-56px, ${
          dimensions.boundedHeight / 2
        }px) rotate(-90deg)`
      }}
    >
      { label }
    </text>
  )}
</g>
)
}

```

How did you do? No worries if you peeked! This code will be here for you if you need to refer back to it when you set up your own charts.

See how we could replicate the d3 axes with a small amount of code? When we know how to do something one way (such as draw axes with a d3 axis generator), this knowledge prevents us from finding another way. D3 has many convenient methods, but they aren't always the best way to draw a chart. In fact, it can often help us to circumvent itself with smaller methods that it uses.

Another benefit of creating our own axes is that we can customize our charts however we want. Want tick marks but no line for your y axes? No problem! We can also style our axes in one place and ensure that our charts are consistent, even when created by different developers.

Set up interactions

In a production app, we would next want to define our interactions. Most charts could benefit from a tooltip - this could be implemented in various ways.

1. We could add a `chart listener rect` to our `Chart` component that would sit on top of its `children`. We could listen to all mouse events and use `d3.scan()` to find the closest point and position the tooltip using our scales (similar to our timeline example in **Chapter 5**).
2. We could add a boolean property to `Line` that creates the listener rect, tying the tooltip to a specific data element. This might be beneficial if we have many types of charts that need different types of listeners (like our scatter plot example in **Chapter 5**).

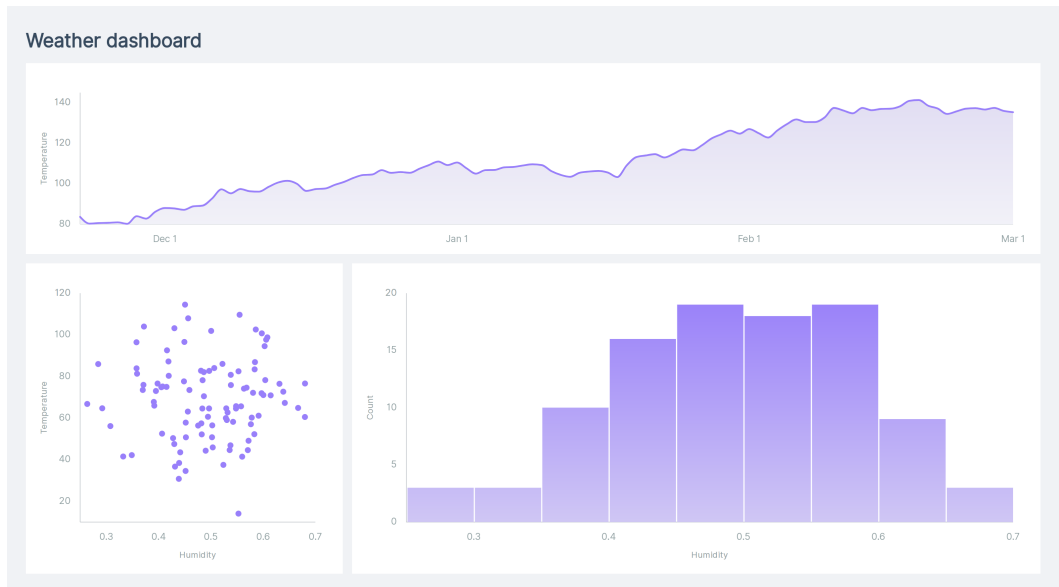
Finishing up

Now that we've created some basic chart components and a **Timeline** component, we have a general idea of how to weave React.js and d3.js together. The general idea is to use React.js for any DOM rendering and d3.js as a utility library.

Populate the rest of the dashboard by switching the import statements in `src/App.jsx` to use the files in the `src/completed/` folder.

```
// import Timeline from "./Timeline"
// import ScatterPlot from "./ScatterPlot"
// import Histogram from "./Histogram"
import Timeline from "./completed/Timeline"
import ScatterPlot from "./completed/ScatterPlot"
import Histogram from "./completed/Histogram"
```

When we look at our browser again, we'll see that the whole dashboard is populated!

**Finished dashboard**

Check out the code in

`src/completed/ScatterPlot.jsx` and

`src/completed/Histogram.jsx`

to see how we converted our d3 code to React + d3 code. For example, instead of using `.enter().append()` we simply map over each item in our dataset.

The completed timeline has an extra area with a gradient fill - check out how that was implemented. One important piece of information to remember here is that our timeline component could appear multiple times on a page, so we need a unique id per gradient instance in order to grab the right one. This is simple enough to implement, but easy to overlook.

If you're feeling comfortable, play around with one of the charts - what if we added a color scale, or sized the circles by a metric? What would it look like to implement a timeline with multiple lines? What about something radically different, like a radar chart? Remember to let React handle the DOM changes and utilize d3 as much as possible for data manipulation and other conveniences like scales.

Using D3 With Angular

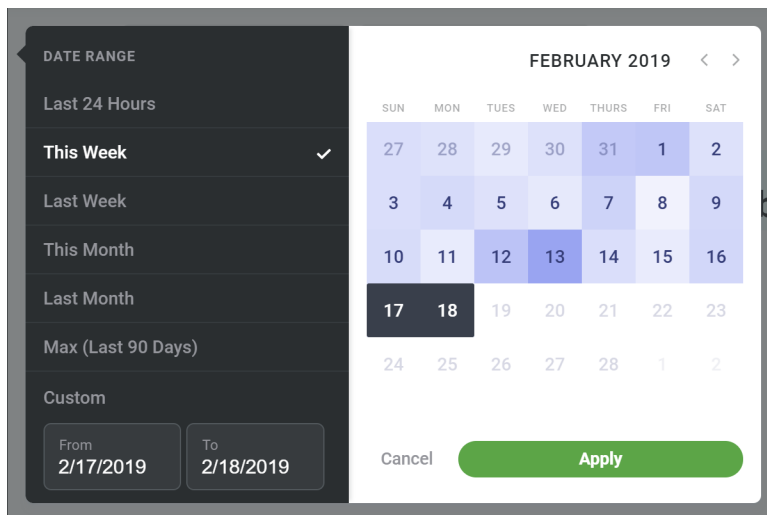
We know how to make individual charts, but you might ask: what's the best way to draw a chart within my current JavaScript framework? D3 can be viewed as a utility library, but it's also used to manipulate the DOM. There is a fair bit of overlap in functionality between a JavaScript framework and d3 — let's talk about the best way to handle that overlap.

First off, we should decide **when** to use d3. Should we use d3 to render a whole page?

Let's split up d3's functionality by concern:

1. DOM manipulation (like jQuery)
2. Data manipulation (manipulation, interpolation, basic stats)

With the library compartmentalized in this way, you might come up with unorthodox ways to utilize d3. For example, I recently used it to create a calendar date picker.



Date picker

This date picker doesn't look like a chart, but d3 came in handy in a few ways.

- **d3-date** helps with splitting up a date range into weeks
- **d3-date** helps with calculating date range defaults — for example, if a user selects **Last Week**, the date picker will use `d3.timeWeek.floor()` and `d3.timeWeek.ceil()` to find the start and end of the week.
- **d3-scale** helps with creating a color scale to show data values between each day. This helps users know which days to select based on the data (in this case, online attention to a specific topic).
- **d3-time-format** helps to display each day's day of the month and the input values on the bottom left.
- **d3-selection** helps create mouse events to select days on hover when a user has selected their start date and will select their end date.

A d3 novice might not think to utilize d3 in this way, but once you've read this book you will be familiar enough to take full advantage of the d3 library.

Angular

Angular is a framework for building modern, component-based user interfaces in HTML and Typescript. Typescript is a superset of Javascript: it looks very similar to Javascript and gets compiled to Javascript, but it has extra features like *static typing*, *classes*, and *interfaces*.

In this chapter, I'm going to focus on the right way to combine Angular and d3.js. Unfortunately, that doesn't leave much space for introducing Angular itself – if you're unfamiliar with Angular, you'll want to spend some time getting familiar first. They have a [quick-start tutorial in their docs](https://angular.io/tutorial)⁸⁴ that might be helpful.

Angular is a large framework that helps to run applications: Because Angular is such a full-featured framework, it doesn't fit nicely with d3. To draw a chart, should we use both of libraries? Just one? Neither?

Having just gotten comfortable with d3, you probably aren't going to like the answer. Instead of using axis generators and letting d3 draw elements, **we're going to let Angular handle the rendering and to use d3 as a (very powerful) utility library**. Let's build an example chart library and see for ourselves why this makes the most sense.

⁸⁴<https://angular.io/tutorial>

Digging in

In the `/code/14-using-d3-with-angular/` folder, you'll find a very bare bones Angular app. Angular comes with a command-line interface tool that helps with various development tasks, like bundling our code. First, let's install that in the terminal with the following command:

```
npm install -g @angular/cli
```

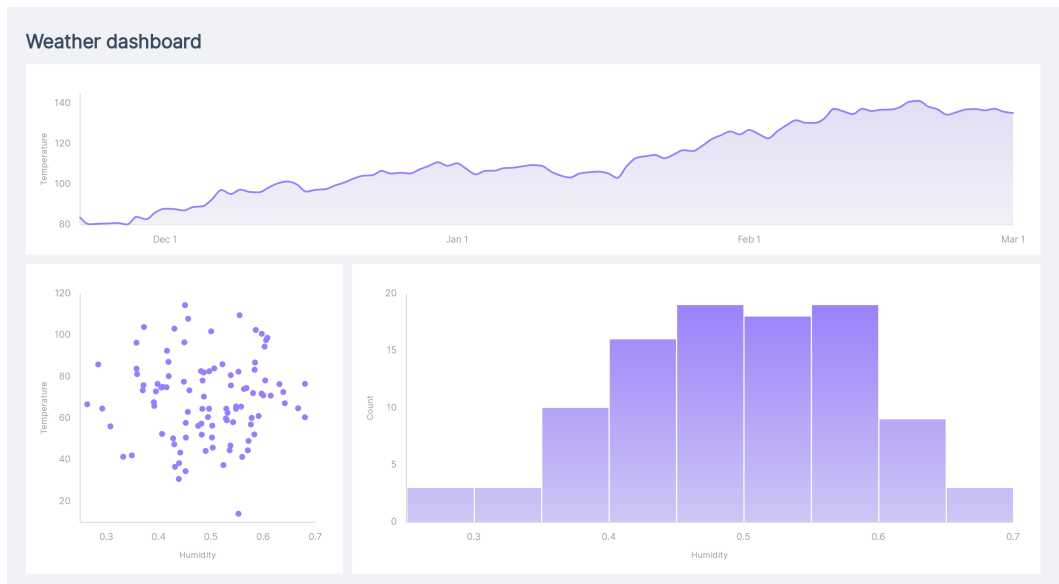
Once npm has finished installing that, let's get our app ready! In the terminal, in the `/code/14-using-d3-with-angular/` folder, download the necessary packages with npm (or yarn, if you prefer).

```
npm install
```

Once your packages are installed, we can run our app:

```
ng serve --open
```

The `--open` parameter will open our app (most likely at <http://localhost:4200>⁸⁵) in the browser, once the server is up and running. You should see an empty dashboard with three placeholders — one for a timeline, one for a scatter plot, and one for a histogram.



Finished dashboard

Within the app folder, we have an app component that is loading random data and updating it every four seconds — this will help us design our chart transitions.

Our chart-making plan has four levels of components:

1. **app**, which will decide what our dataset is and how to access values for our axes (accessors),
2. **timeline**, **scatter**, or **histogram** which will be re-useable components that decide how a specific type of chart is laid out and what goes in it,

⁸⁵<http://localhost:4200>

3. **chart**, which will render our **wrapper** and **bounds**, and
4. **axis**, **line**, **bars**, etc., which will create individual elements within our charts.

Levels 3 and 4 will be isolated in the `chart` folder, creating a charting library that can be used throughout the app to make many different types of charts. Having a basic charting library will help in many ways, such as abstracting svg components idiosyncracies (for example, collaborators won't need to know that you need to use a `<rect>` to create a rectangle, and it takes a `x` attribute whereas a `<circle>` takes a `cx` attribute).

If you're feeling lost or want to see the finished code, the completed charts and chart library are over in `/code/14-using-d3-with-angular/app/completed` to help you out.

We'll start by fleshing out our `timeline` component, running through our usual chart making steps.

1. **Access data**
2. **Create dimensions**
3. **Draw canvas**
4. **Create scales**
5. **Draw data**
6. **Draw peripherals**
7. **Set up interactions**

Access data

Let's open up our `timeline` component, located in

`src/app/timeline/timeline.component.ts`. There isn't much in here: the bones of an Angular component, types for our inputs, and all of the imports we'll need.

code/14-using-d3-with-angular/src/app/timeline/timeline.component.ts

```
6 @Component({
7   selector: 'app-timeline',
8   templateUrl: './timeline.component.html',
9   styleUrls: ['./timeline.component.css'],
10 })
11 export class TimelineComponent {
12   @Input() data: object[]
13   @Input() label: string
14   @Input() xAccessor: AccessorType
15   @Input() yAccessor: AccessorType
16
17 }
```

The `@Component` object at the top helps configure the different parts of our component:

1. the `selector` sets how to create an instance of our component. If we look in `app.component.html`, we'll see that the template uses this tag to create a new `timeline`:

code/14-using-d3-with-angular/src/app/app.component.html

```
5     <app-timeline
6       [data]="timelineData"
7       [xAccessor]="dateAccessor"
8       [yAccessor]="temperatureAccessor"
9       label="Temperature">
10   </app-timeline>
```

We can also see the different parameters that `app` is passing to our `timeline` – we'll look at those in more depth in a minute.

Notice that we're wrapping some of our `app-timeline`'s attributes in square brackets (`[]`) – this specifies that we want to pass the *variable with that name* instead of the string. For example, we're passing the `timelineData` variable (defined as `this.timelineData` in `app.component.ts`), instead of the string `"timelineData"`.

1. the `templateUrl` key tells our component where to find this component's template. We could also define our template inline, using a plain string, which we'll take advantage of later.
2. the `styleUrls` key tells our component where to find our component styles. If we look in the `timeline.component.css` file, we'll see that our styles are already populated for us – this is to help us to focus on the Angular code. If at any point you're wondering *why* our components look the way they do, the answer is in the linked `.css` files!

Our `timeline` component takes four properties:

1. `data`
2. `xAccessor`
3. `yAccessor`
4. `label`

These properties are flexible enough to support throwing a timeline anywhere in our dashboard with any dataset. But we don't have so many properties that creating a new timeline is overwhelming or allows us to create inconsistent timelines.

Create dimensions

Next up, we need to specify the size of our chart. In our dashboard, we could have timelines of many different sizes. Each of these timelines are also likely to change size based on the window size. To keep things flexible, we'll need to grab the dimensions of our container `<div>`.

Let's first add a new public variable named `dimensions` – we can use a custom type that we've imported from `/chart/utils.ts`. If we look inside `/chart/utils.ts`, we can see our `dimensions` object is expected to have a `height`, `width`, and all four margins.

`code/14-using-d3-with-angular/src/app/completed/chart/utils.ts`

```
5 export interface DimensionsType {  
6   marginTop: number  
7   marginRight: number  
8   marginBottom: number  
9   marginLeft: number  
10  height: number  
11  width: number  
12  boundedHeight?: number  
13  boundedWidth?: number  
14 }
```

To keep things simple, this object is flat, unlike some `dimensions` objects we've used before. In practice, if you need to rely on a specific structure for your `dimensions` object, it might be better to keep it flat instead of nesting margins inside another object.

Let's get back to our `timeline.component.ts` file and define our `dimensions` variable.

`code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts`

```
16 public dimensions: DimensionsType
```

Next, within our constructor, we'll create our `dimensions`.

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
27   constructor() {
28     this.dimensions = {
29       marginTop: 40,
30       marginRight: 30,
31       marginBottom: 75,
32       marginLeft: 75,
33       height: 300,
34       width: 600,
35     }
36     this.dimensions = {
37       ...this.dimensions,
38       boundedHeight: Math.max(this.dimensions.height
39         - this.dimensions.marginTop
40         - this.dimensions.marginBottom, 0),
41       boundedWidth: Math.max(this.dimensions.width
42         - this.dimensions.marginLeft
43         - this.dimensions.marginRight, 0),
44     }
45   }
```

While we want to specify the exact height of our timeline, we want our timeline to stretch horizontally to fit its container. After we've defined our dimensions, let's use `@ViewChild()` to create an `ElementRef` and hook onto an element in our template.

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
25   @ViewChild('container', {static: true}) container: ElementRef
```

Starting with Angular 8, we need to add a second parameter to `@ViewChild`, specifying that `static` is `true`. This setting ensures that our container is ready by the time we need to use it (in the `ngAfterContentInit` hook, which we'll talk about in a minute).

Read more [about this flag^a](https://angular.io/guide/static-query-migration) or [about ElementRef^b](https://angular.io/api/core/ViewChild) on the Angular docs.

^a<https://angular.io/guide/static-query-migration>

^b<https://angular.io/api/core/ViewChild>

Jumping over to our timeline's template file (`timeline.component.html`), we can add `#container` to mark our outside `<div>` as the element we're concerned with.

`code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.html`

1 `<div #container>`

Back in our `timeline.component.ts` file, at the bottom of our `TimelineComponent` class, let's add a function that updates our dimensions, based on the width of our `#container <div>`. We can access our container element using

`this.container.nativeElement`.

```
updateDimensions() {  
  const width = this.container.nativeElement.offsetWidth  
  this.dimensions.width = width  
  this.dimensions.boundedWidth = Math.max(  
    this.dimensions.width  
    - this.dimensions.marginLeft  
    - this.dimensions.marginRight,  
    0  
  )  
  console.log(dimensions)  
}
```

We'll want to execute our `updateDimensions()` function *directly after our component is first rendered*. We'll use the `ngAfterContentInit` lifecycle hook – read about it and others [in the Angular docs⁸⁶](https://angular.io/guide/lifecycle-hooks).

First, we'll need to specify that we want to use `ngAfterContentInit` when we create our class.

⁸⁶<https://angular.io/guide/lifecycle-hooks>

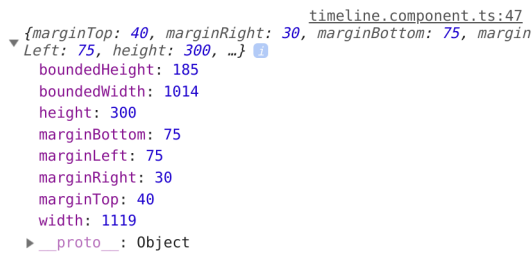

```
export class TimelineComponent implements AfterContentInit {
```

Now we can add a new function to the bottom of our TimelineComponent definition.

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
63   ngAfterContentInit() {  
64     this.updateDimensions()  
65   }
```

Now when we load our dashboard in the browser, we'll see our dimensions object logged in our Dev Tools console.

A screenshot of the Chrome DevTools console. It shows a log entry for a JavaScript object. The object has properties: marginTop: 40, marginRight: 30, marginBottom: 75, marginLeft: 75, height: 300, boundedHeight: 185, boundedWidth: 1014, width: 1119. The width property is highlighted in blue. The log entry is from timeline.component.ts:47. Below the object, it shows __proto__: Object.

```
timeline.component.ts:47  
{marginTop: 40, marginRight: 30, marginBottom: 75, margin  
Left: 75, height: 300, ...}  
  boundedHeight: 185  
  boundedWidth: 1014  
  height: 300  
  marginBottom: 75  
  marginLeft: 75  
  marginRight: 30  
  marginTop: 40  
  width: 1119  
  __proto__: Object
```

Dimensions object in console

We can tell that we're updating our width because dimensions.width has updated from its original value of 600 pixels.

Updating dimensions on window resize

Our dimensions fit our container <div> when initialized, but how do we update them when our window is resized?

Thankfully, Angular makes these kinds of event listeners very easy. We can use a [@HostListener](https://angular.io/api/core/HostListener)⁸⁷ to run our updateDimensions() function every time the window resizes.

⁸⁷<https://angular.io/api/core/HostListener>

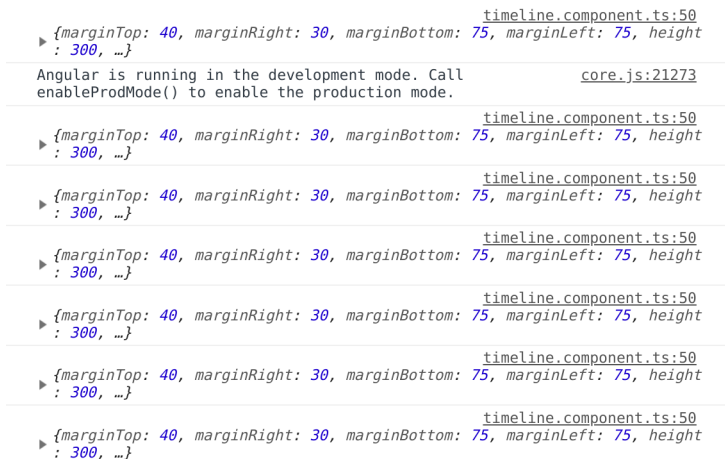
code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```

47   @HostListener('window:resize') windowResize() {
48       this.updateDimensions()
49   }

```

Great! Now when we resize our window, we'll see our dimensions object with an updated width number in the Dev Tools console.



The screenshot shows the Chrome DevTools console with several log entries. Each entry displays a JavaScript object representing dimensions: {marginTop: 40, marginRight: 30, marginBottom: 75, marginLeft: 75, height: 300, ...}. The source for each log is 'timeline.component.ts:50'. Between the first and second log entries, there is a message: 'Angular is running in the development mode. Call enableProdMode() to enable the production mode.' from 'core.js:21273'. The logs show that the dimensions object is being updated repeatedly as the window is resized.

Dimensions object in console, on resize

Draw canvas

Next up, we need to create our `<svg>` element. Since we'll want a canvas for all of our charts, we'll wrap this work into a component that we can re-use. Let's open up the `/charts/chart.component.ts` file.

We don't need much to happen here, but we do want to pass our chart a dimensions object so it can shift our chart **bounds**. Let's define our dimensions with our custom `dimensionsType` (which is already imported into this file).

code/14-using-d3-with-angular/src/app/completed/chart/chart.component.ts

```
4 @Component({  
5   selector: 'app-chart',  
6   templateUrl: './chart.component.html',  
7   styleUrls: ['./chart.component.css']  
8 })  
9 export class ChartComponent {  
10   @Input() dimensions: DimensionsType  
11 }
```

Next, we'll switch over to the template file at `chart/chart.component.html` to add our **wrapper** and **bounds**.

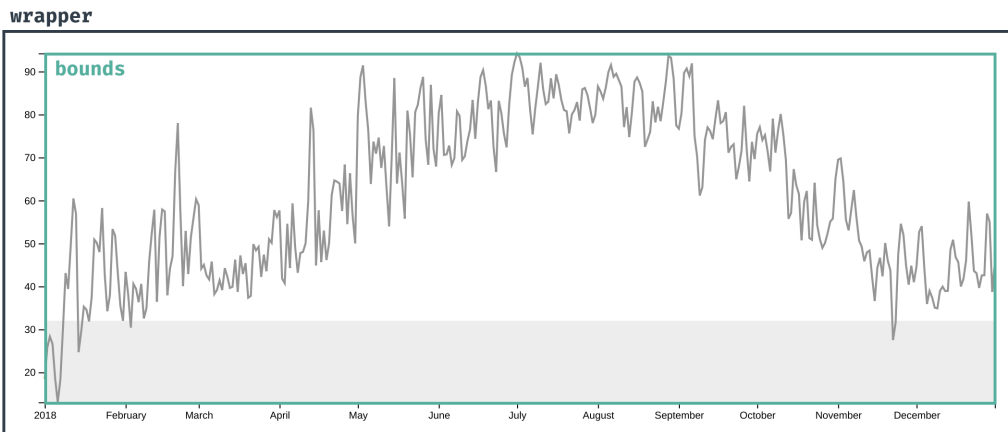


Chart dimensions

First, we'll add our **wrapper**: a `<svg>` element sized with our dimensions. We can set our HTML attributes using `attr.`, and we'll wrap our `[attr.]` statement in square brackets `[]` to specify that we want to pass a variable name, instead of a plain string.

```
<svg [attr.height]="dimensions.height" [attr.width]="dimensions.width">
</svg>
```

Next, we'll add our **bounds**: a `<g>` element that is shifted by our top and left margins. If we use a plain `<g>` tag, Angular will think we're using a custom directive. To tell Angular that we're using the SVG namespace, we'll need to add a `svg:` prefix to all of our SVG elements' tags.

```
<svg [attr.height]="dimensions.height" [attr.width]="dimensions.width">
  <svg:g>
  </svg:g>
</svg>
```

Now we can shift our `<g>` element using our top and left margins. We'll use a `style` prefix to specify that we want our transform to be applied as a CSS style.

```
<svg [attr.height]="dimensions.height" [attr.width]="dimensions.width">
  <svg:g [style.transform]="[
    'translate(',
    dimensions.marginLeft,
    'px, ',
    dimensions.marginTop,
    'px)'
  ].join('')">
  </svg:g>
</svg>
```

Note that we're `.join()`ing an array of strings to build our transform string. This helps us keep our templates readable. We would normally use an ES6 template literal (`${}`), but angular templates don't recognize that grammar.

When we use our chart component, we'll want to nest our data and peripheral elements inside. Let's add a `<ng-content />` tag to pass any nested elements into our **bounds**.

code/14-using-d3-with-angular/src/app/completed/chart/chart.component.html

```
1 <svg [attr.height]="dimensions.height" [attr.width]="dimensions.width">
2   <svg:g [style.transform]="[
3     'translate(',
4     dimensions.marginLeft,
5     'px, ',
6     dimensions.marginTop,
7     'px) '
8   ].join('')">
9     <ng-content></ng-content>
10   </svg:g>
11 </svg>
```

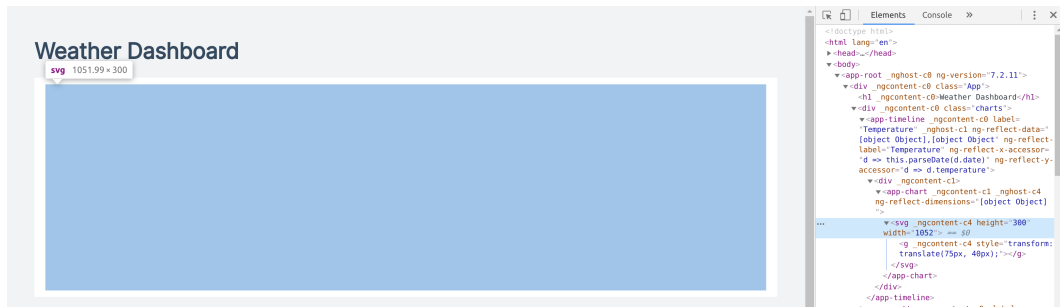
Using our chart component

If we look at our webpage, nothing has changed. We'll need to add a chart inside of our timeline component. Let's switch to our `timeline/timeline.component.html` file and add a chart inside of our container element.

```
<div #container>
  <app-chart
    [dimensions]="dimensions">
  </app-chart>
</div>
```

We're using the `app-chart` tag name here, since that's what we specified as our chart component's selector in the configuration object.

Perfect! We can now see a `<svg>` and `<g>` element in the **Elements** panel of our dev tools. When we resize our window, we'll see that our `<svg>` updates its size to fit perfectly



`<svg>` element in console

Create scales

Next up, we need to create the scales to convert from the data domain to the pixel domain. Let's pop back to our timeline's typescript file:

`timeline/timeline.component.ts` and add some definitions.

We'll need an `x` and a `y` scale, as well as accessor functions.

`code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts`

```

17  public xScale: ScaleType
18  public yScale: ScaleType
19  public xAccessorScaled: AccessorType
20  public yAccessorScaled: AccessorType
21  public y0AccessorScaled: AccessorType

```

At the end of our `timeline` class, let's create an `updateScales()` function that creates our scales. We'll also make **scaled accessor functions** that we can pass to chart components, so that they don't need to be aware of our scales.

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
71   updateScales() {
72     this.xScale = d3.scaleTime()
73       .domain(d3.extent(this.data, this.xAccessor))
74       .range([0, this.dimensions.boundedWidth])
75
76     this.yScale = d3.scaleLinear()
77       .domain(d3.extent(this.data, this.yAccessor))
78       .range([this.dimensions.boundedHeight, 0])
79       .nice()
80
81     this.xAccessorScaled = d => this.xScale(this.xAccessor(d))
82     this.yAccessorScaled = d => this.yScale(this.yAccessor(d))
83     this.y0AccessorScaled = this.yScale(this.yScale.domain()[0])
84   }
```

When to update our scales?

Because our scales use our dimensions, we'll want to update them whenever our dimensions change. Let's run our `updateScales()` function at the end of our `updateDimensions()` function. Because `updateScales()` runs when our component is initialized, this takes care of creating our scales the first time.

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
51   updateDimensions() {
52     const width = this.container.nativeElement.offsetWidth
53     this.dimensions.width = width
54     this.dimensions.boundedWidth = Math.max(
55       this.dimensions.width
56       - this.dimensions.marginLeft
57       - this.dimensions.marginRight,
58       0
59     )
60     this.updateScales()
61   }
```

We also want our scales to update whenever our data changes. We can use the `ngOnChanges` lifecycle hook. First, let's tell our component that we're going to use the hook:

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
11 export class TimelineComponent implements AfterContentInit, OnChanges {
```

Then we'll add a `ngOnChanges()` function to our class definition, specifying that we want it to run on `SimpleChanges`.

code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts

```
67   ngOnChanges(changes: SimpleChanges): void {  
68     this.updateScales()  
69   }
```

If you're curious, you can [read more about this hook in the Angular docs](https://angular.io/api/core/OnChanges)^a.

^a<https://angular.io/api/core/OnChanges>

Draw data

Here's the best step! Now we get to draw our data elements – in this case, a line. Let's prepare by creating a `line` component. Let's open up the

`chart/line/line.components.ts` file, which has the bones of a new component.

code/14-using-d3-with-angular/src/app/chart/line/line.component.ts

```
4 @Component({  
5   selector: '[appLine]',  
6   template: ``,  
7   styleUrls: ['./line.component.css']  
8 })  
9 export class LineComponent {  
10 }
```

There are two differences with this component:

1. We're defining our `selector` using square brackets (`[]`). This makes it an **attribute selector**, which specifies that our component must be used as an **attribute on another element**. For example, we'll need to use our `line` component with the code `<svg:g app-line />`, and not `<app-line />`.
2. Instead of passing in a `templateUrl` that links to another `.html` file, we'll be defining our `template` inline. This will keep our code concise, since our `line` template will only be a few lines.

We could use a `@Directive` instead of a `@Component` here. The difference is that a directive will alter an externally created DOM element.

Using a directive would force anyone who wants to use our `line` component to know that they need to use a `<path>` element. We'll instead use a component so that all of our chart elements can be created with a `<svg:g>` element.

Let's start fleshing our component out, starting with defining any properties we'll want to pass to it.

code/14-using-d3-with-angular/src/app/completed/chart/line/line.component.ts

```

17   @Input() type: "area" | "line" = "line"
18   @Input() data: object[]
19   @Input() xAccessor: AccessorType
20   @Input() yAccessor: AccessorType
21   @Input() y0Accessor?: AccessorType
22   @Input() interpolation?: Function = d3.curveMonotoneX
23   @Input() fill?: string

```

We'll also want to define a `lineString` variable that will hold our `d` attribute string that will tell the line what shape to be.

code/14-using-d3-with-angular/src/app/completed/chart/line/line.component.ts

```

24   private lineString: ""

```

Next, we'll make a function that will create our `lineString` variable. We'll keep our component flexible, and handle drawing either a line or an area. `d3.line()` and `d3.area()` are very similar, except that `d3.line()` uses a `.y()` method and `d3.area()` uses a `.y0()` and a `.y1()` method (for the bottom and top of the area).

code/14-using-d3-with-angular/src/app/completed/chart/line/line.component.ts

```

26   updateLineString(): void {
27     const lineGenerator = d3[this.type]()
28       .x(this.xAccessor)
29       .y(this.yAccessor)
30       .curve(this.interpolation)
31
32     if (this.type == "area") {
33       lineGenerator
34         .y0(this.y0Accessor)
35         .y1(this.y1Accessor)
36     }
37
38     this.lineString = lineGenerator(this.data)
39   }

```

We want to execute this function any time any of our properties changes. At the top of our component definition, let's tell our component that we want to use an `OnChanges` lifecycle hook.

code/14-using-d3-with-angular/src/app/completed/chart/line/line.component.ts

```
16 export class LineComponent implements OnChanges {
```

And at the bottom of our component definition, we'll add an `ngOnChanges()` function that will run our `updateLineString()` function.

code/14-using-d3-with-angular/src/app/completed/chart/line/line.component.ts

```
41   ngOnChanges(changes: SimpleChanges): void {  
42     this.updateLineString()  
43   }
```

Lastly, we'll update our template string in our `@Component` configuration. We want to draw one `<svg:path>` element and set its `d` attribute to our `lineString`. We'll also want to give it a class, linked to our `line`'s type property, so we can style our `line` and `area` types differently, and pass down a `fill` style, which will come in handy if we want to pass a specific fill color or gradient id.

code/14-using-d3-with-angular/src/app/completed/chart/line/line.component.ts

```
5 @Component({  
6   selector: '[appLine]',  
7   template: `  
8     <svg:path  
9       [ngClass]="type"  
10      [attr.d]="lineString"  
11      [style.fill]="fill">  
12     </svg:path>  
13   `,  
14   styleUrls: ['./line.component.css']  
15 })
```

Using our line component

Hopping back to our `timeline/timeline.component.html` file, let's create an instance of our line component. For all of our chart components, we'll initialize them as **attributes of a `<g>` element**. This will keep our chart library's API as simple as possible.

We'll also pass our line element our data and scaled x and y accessors.

```
<div #container>
  <app-chart
    [dimensions]="dimensions">
    <svg:g appLine
      [data]="data"
      [xAccessor]="xAccessorScaled"
      [yAccessor]="yAccessorScaled">
    </svg:g>
  </app-chart>
</div>
```

Nice! Now when we look at our webpage, we can see a squiggly line that updates every few seconds.



chart with line

Draw peripherals

Next, we want to draw our axes. This is where even experienced d3.js and Angular developers get confused because both libraries want to handle creating new the DOM elements. Up until now, we've used `d3.axisBottom()` and `d3.axisLeft()` to append multiple `<line>` and `<text>` elements to a manually created `<g>` element. But because Angular is such a full-featured framework, it's important to let it have full control over the DOM.

Let's first make a naive attempt at an `axis` component, mimicking the d3.js code we've written so far. Since we're already in our `timeline/timeline.component.html` file, let's start by adding a new `<svg:g>` element with an attribute of `appAxis` (assuming that the API will be the same as our `line` component). We'll make this a `y` axis by giving it a dimension of `"y"`, and we'll pass other variables that we'll need: `dimensions`, `scale`, and a `label`.

```
<div #container>
  <app-chart
    [dimensions]="dimensions">
    <svg:g appAxis
      dimension="y"
      [dimensions]="dimensions"
      [scale]="yScale"
      [label]="label">
    </svg:g>
    <svg:g appLine
      [data]="data"
      [xAccessor]="xAccessorScaled"
      [yAccessor]="yAccessorScaled">
    </svg:g>
  </app-chart>
</div>
```

Remember that SVG elements' z-indices are determined by their order in the DOM. If you want your line to overlap your axes, make sure to add the `appAxis` component before the `appLine` component.

Let's start working on our `axis` component in the `chart/axis/axis.component.ts` file. We can see that we're already started with a component definition that looks very similar to our `line` component.

`code/14-using-d3-with-angular/src/app/chart/axis/axis.component.ts`

```
5 @Component({
6   selector: '[appAxis]',
7   templateUrl: './axis.component.html',
8   styleUrls: ['./axis.component.css']
9 })
10 export class AxisComponent {
11 }
```

First, we'll define all of the properties our component will handle.

`code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.ts`

```
11 @Input() dimensions: DimensionsType
12 @Input() dimension: "x" | "y" = "x"
13 @Input() scale: ScaleType
14 @Input() label: string
```

In order to use `d3.select()` to grab an element, we'll need to use an `ElementRef`. Let's create a new `ElementRef` named `axis`.

```
@ViewChild('axis', {static: true}) axis: ElementRef
```

And in the `chart/axis/axis.component.html` file, we'll tag a new `<g>` element with `#axis`, telling Angular that this is the DOM element we want to target.

```
<svg:g #axis>
</svg:g>
```

Next, we'll create a function at the end of our class definition in `chart/axis/axis.component.ts` that generates a d3 axis on our targeted element.

```
updateTicks() {
  const yAxisGenerator = d3.axisLeft()
    .scale(this.scale)

  const yAxis = d3.select(this.axis.nativeElement).call(yAxisGenerator)

  d3.select(this.axis.nativeElement)
}
```

We'll want to use a different method based on the location of our axis (`d3.axisLeft()` or `d3.axisBottom()`), but we'll keep things simple for now, since this is an example.

We want our `updateTicks()` function to run whenever a property updates, so we'll tell Angular that we'll want an `OnChanges` hook (at the top of our file).

`code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.ts`

```
10 export class AxisComponent implements OnChanges {
```

And then we'll create that function at the bottom of our file.

code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.ts

```
30   ngOnChanges(changes: SimpleChanges): void {  
31     this.updateTicks()  
32   }
```

Great! Now when we load our website, our timeline will have a y axis that updates with our line.



Timeline with a y axis

Axes, take two

Now that we've shown how easy it is to plug basic d3 code into Angular, let's talk about **why it's not a good idea**.

One of the benefits of Angular is that it processes each component's template code and optimizes it: cutting down on CPU cycles, memory consumption, and garbage collection. **When we mutate the DOM outside of Angular templates, we're bypassing these performance benefits.**

Additionally, manually DOM manipulations aren't guaranteed to work with other benefits of Angular, such as its [server-side rendering solution: Universal](https://angular.io/guide/universal)⁸⁸.

Short-cutting around Angular render methods also makes our code less declarative. Usually, you can depend on the resulting DOM to closely align to a component's

⁸⁸<https://angular.io/guide/universal>

template code. When you come back to a component you wrote a few months ago, you'll thank yourself for making the output obvious.

In a pinch, using Angular to create a wrapper element to modify with d3 (like we just did) will do. You might need to do this in special situations, like animating an arc. But try to lean towards solely creating elements with Angular and using d3 as more of a utility library. This will keep your app speedy and less “hacky”.

Even without its DOM modifying methods, d3 is a very powerful library. In fact, we created most of our timeline without needing to re-create a d3 generator function. For example, creating a d string for our `line` component would have been tricky without `d3.line()`.

But how does this look in practice? Let's re-create our `axis` component without using any axis generators.

Re-creating our `axis` component

Instead of using a d3 axis generator in our `chart/axis/axis.component.ts` file, we'll be generating an array of ticks and creating a `<text>` element for each one. Let's start by defining a new `ticks` variable:

`code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.ts`

```
16  private ticks: Function[]
```

Next, we'll need to update our `updateTicks()` function. Fortunately, many of the methods d3 uses internally are also available for external use. d3 scales have a `.ticks()` method that will create an array with evenly spaced values in the scale's domain.

By default, `.ticks()` will aim for ten ticks, but we can pass a specific count to target. Note that `.ticks()` will *aim for* the count, but also tries to create ticks with meaningful intervals: for example, a week in a time scale.

The number of ticks we want will depend on the chart width, though — ten ticks will likely crowd our y axis. Let's aim for one tick per 100 pixels for small screens and one tick per 250 pixels for wider screens on the x axis, and one tick per 70 pixels on the y axis. You'll want to play around with these numbers yourself — they will depend on your preference and font-sizes.

code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.ts

```

18   updateTicks() {
19       if (!this.dimensions || !this.scale) return
20
21       const numberOfTicks = this.dimension == "x"
22           ? this.dimensions.boundedWidth < 600
23           ? this.dimensions.boundedWidth / 100
24           : this.dimensions.boundedWidth / 250
25           : this.dimensions.boundedHeight / 70
26
27       this.ticks = this.scale.ticks(numberOfTicks)
28   }

```

We'll need to turn our raw tick numbers into human-friendly labels - we'll add a `formatTick` input near the top of our class so that the parent chart can specify the formatting. This will be a function that takes a tick value and converts it into a human-readable string.

code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.ts

```

15   @Input() formatTick: (value: any) => (string|number) = d3.format(",")

```

Note that we set a default value for our dimension and `formatTick` inputs. This way, we won't need to specify these values every time we create an `Axis`.

Moving over to `chart/axis/axis.component.html`, we'll want to update our template code to map over our `ticks`. Since the position of our axis components will vary based on the axis type (x or y), let's create those two types separately.

First, we'll focus on the x type, using `*ngIf` to only render these elements if our dimension property equals "x". Our axis will be composed of:

1. a `<line>` to show the bounds of our axis,
2. one `<text>` element for each item in our `ticks` array, and
3. a `<text>` label, if we have a `label` property, positioned just below our axis, centered horizontally.

code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.html

```
1 <svg:g
2   *ngIf="dimension == 'x'"
3   class="x"
4   attr.transform="translate(0, {{dimensions.boundedHeight}})">
5   <svg:line
6     [attr.x2]="dimensions.boundedWidth">
7   </svg:line>
8   <svg:text
9     *ngFor="let tick of ticks"
10    class="tick"
11    attr.transform="translate({{scale(tick)}}, 25)">
12    {{ formatTick(tick) }}
13  </svg:text>
14  <svg:text
15    *ngIf="label"
16    class="label"
17    attr.transform="translate({{dimensions.boundedWidth / 2}}, 60)">
18    {{ label }}
19  </svg:text>
20 </svg:g>
```

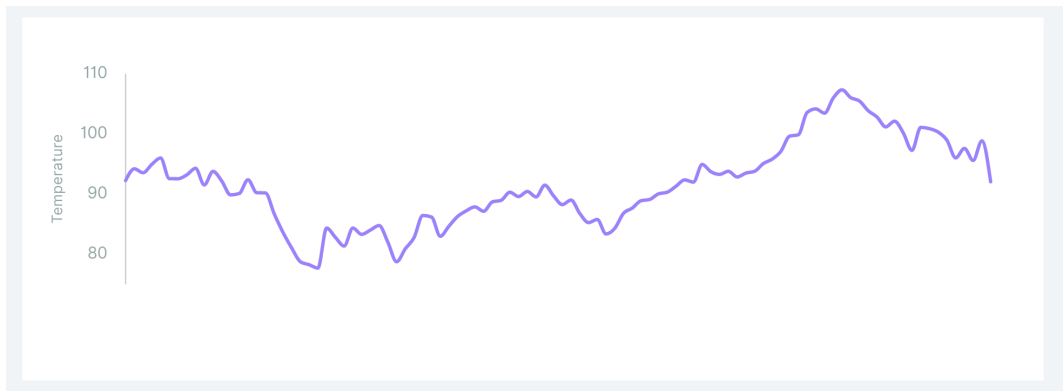
Remember that `<line>` elements are positioned with `x1`, `x2`, `y1`, and `y2` attributes. We'll want to draw a line from `[0,0]` to `[dimensions.boundedWidth, 0]` — since `x1`, `x2`, and `y1` will all be 0 (the default), we can leave those attributes out.

Our y axis will be very similar — we'll create the same elements, but position them differently. Note that we'll rotate our label `<text>` 90 degrees counter-clockwise.

code/14-using-d3-with-angular/src/app/completed/chart/axis/axis.component.html

```
21 <svg:g
22   class="y"
23   *ngIf="dimension == 'y'">
24   <svg:line
25     [attr.y2]="dimensions.boundedHeight">
26   </svg:line>
27   <svg:text
28     *ngFor="let tick of ticks"
29     class="tick"
30     attr.transform="translate(-16, {{scale(tick)}})">
31     {{ formatTick(tick) }}
32   </svg:text>
33   <svg:text
34     *ngIf="label"
35     class="label"
36     [ngStyle]="{transform: [
37       'translate(-56px, ',
38       dimensions.boundedHeight / 2,
39       'px) rotate(-90deg) '
40     ].join('')}">
41     {{ label }}
42   </svg:text>
43 </svg:g>
```

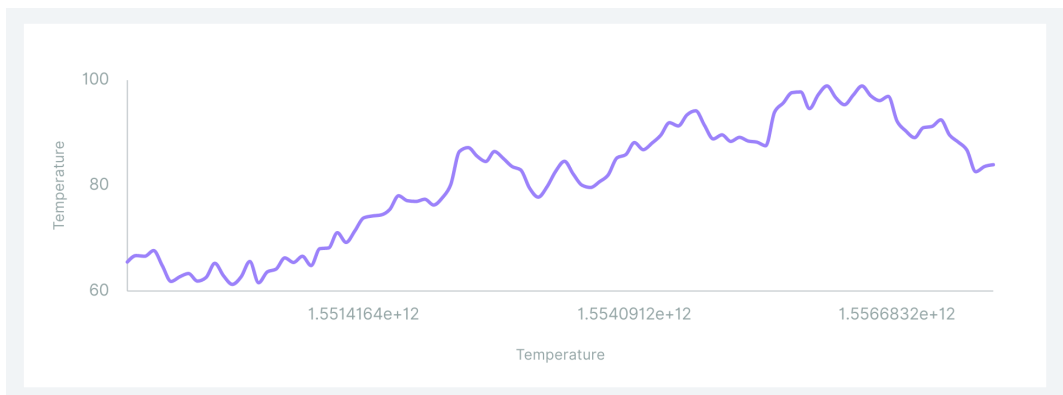
That was a mouthful! We ended up writing way more template code – partially because we were handling both x and y axis types, but mostly because we’re not relying on d3 to create all of these elements for us. Cheer up, though! The axis component is the worst part of using Angular and d3 this way.

**Timeline with a y axis**

We're missing our x axis – let's hop back over to `timeline/timeline.component.html` and add that next to our y axis.

```
<svg:g appAxis
  dimension="x"
  [dimensions]="dimensions"
  [scale]="xScale"
  [label]="label">
</svg:g>
```

When we look at our chart, we can see a wonderful x axis with ticks:

**Timeline with an x axis**

Those dates don't look right, though. d3 axis generators have built-in logic that will detect date strings and format them correctly.

Let's override the default `formatTick` prop and pass a `formatDate` function that we'll define in a minute.

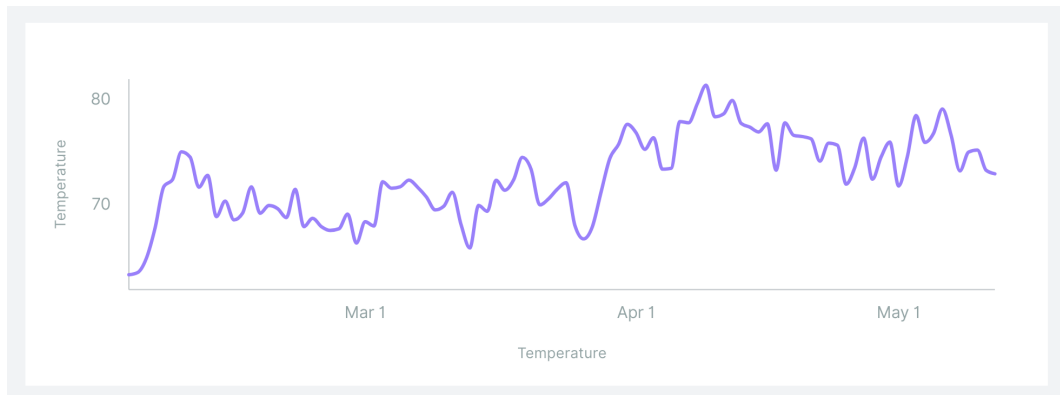
```
<svg:g appAxis
  dimension="x"
  [dimensions]="dimensions"
  [scale]="xScale"
  [label]="label"
  [formatTick]="formatDate">
</svg:g>
```

In our `timeline/timeline.component.ts` file, we'll create our `formatDate` function using `d3.timeFormat()`.

`code/14-using-d3-with-angular/src/app/completed/timeline/timeline.component.ts`

```
22  public formatDate: (date: object) => string = d3.timeFormat("%-b %-d")
```

That's much better!



Timeline with an x axis, with formatted dates

In your own Angular chart library, it might be a good idea to detect whether or not the tick is a date object and format it accordingly. That will depend on your use cases:

how often will you need to format dates? Will you want all dates to be formatted the same way?

See how we could replicate the `d3` axes with a small amount of code? When we know how to do something one way (such as draw axes with a `d3` axis generator), this knowledge prevents us from finding another way. `D3` has many convenient methods, but they aren't always the best way to draw a chart. In fact, it can often help us to circumvent itself with smaller methods that it uses.

Another benefit of creating our own axes is that we can customize our charts however we want. Want tick marks but no line for your `y` axes? No problem! We can also style our axes in one place and ensure that our charts are consistent, even when created by different developers.

Set up interactions

In a production app, we would next want to define our interactions. Most charts could benefit from a tooltip - this could be implemented in various ways.

1. We could add a chart listener rect to our `chart` component that would sit on top of its nested elements. We could listen to all mouse events and use `d3.scan()` to find the closest point and position the tooltip using our scales (similar to our timeline example in **Chapter 5**).
2. We could add a boolean property to our `line` component that creates the listener rect, tying the tooltip to a specific data element. This might be beneficial if we have many types of charts that need different types of listeners (like our scatter plot example in **Chapter 5**).

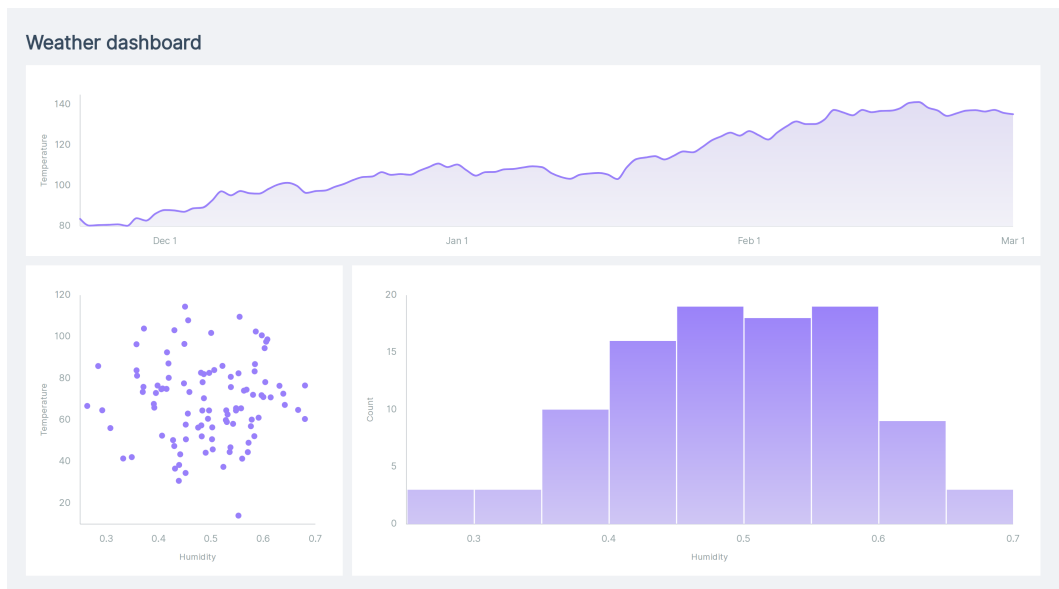
Finishing up

Now that we've created some basic chart components and a `timeline` component, we have a general idea of how to weave Angular and `d3.js` together. The general idea is to use Angular for any DOM rendering and `d3.js` as a utility library.

Populate the rest of the dashboard by switching the import statements in `src/app/app.module.ts` to use the files in the `src/completed/` folder.

```
import { AppComponent } from './app.component'
// import { TimelineComponent } from './timeline/timeline.component'
// import { ScatterComponent } from './scatter/scatter.component'
// import { HistogramComponent } from './histogram/histogram.component'
// import { ChartComponent } from './chart/chart.component'
// import { AxisComponent } from './chart/axis/axis.component'
// import { LineComponent } from './chart/line/line.component'
// import { CirclesComponent } from './chart/circles/circles.component'
// import { BarsComponent } from './chart/bars/bars.component'
// import { GradientComponent }
//   from './chart/gradient/gradient.component'
import { TimelineComponent }
  from './completed/timeline/timeline.component'
import { ScatterComponent }
  from './completed/scatter/scatter.component'
import { HistogramComponent }
  from './completed/histogram/histogram.component'
import { ChartComponent }
  from './completed/chart/chart.component'
import { AxisComponent }
  from './completed/chart/axis/axis.component'
import { LineComponent }
  from './completed/chart/line/line.component'
import { CirclesComponent }
  from './completed/chart/circles/circles.component'
import { BarsComponent }
  from './completed/chart/bars/bars.component'
import { GradientComponent }
  from './completed/chart/gradient/gradient.component'
```

When we look at our browser again, we'll see that the whole dashboard is populated!

**Finished dashboard**

Check out the code in

`src/app/completed/scatter.component.ts` and

`src/app/completed/histogram.component.ts`

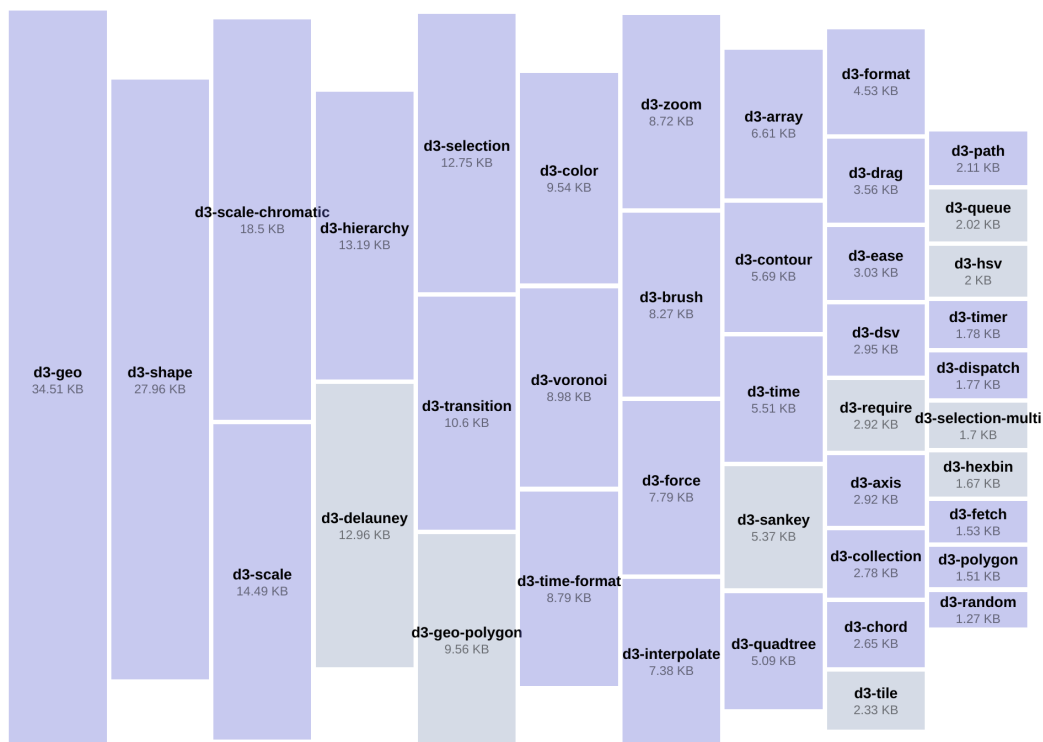
to see how we converted our d3 code to Angular + d3 code. For example, instead of using `.enter().append()` we simply map over each item in our dataset.

The completed timeline has an extra area with a gradient fill - check out how that was implemented. One important piece of information to remember here is that our timeline component could appear multiple times on a page, so we need a unique id per gradient instance in order to grab the right one. This is simple enough to implement, but easy to overlook.

If you're feeling comfortable, play around with one of the charts - what if we added a color scale, or sized the circles by a metric? What would it look like to implement a timeline with multiple lines? What about something radically different, like a radar chart? Remember to let Angular handle the DOM changes and utilize d3 as much as possible for data manipulation and other conveniences like scales.

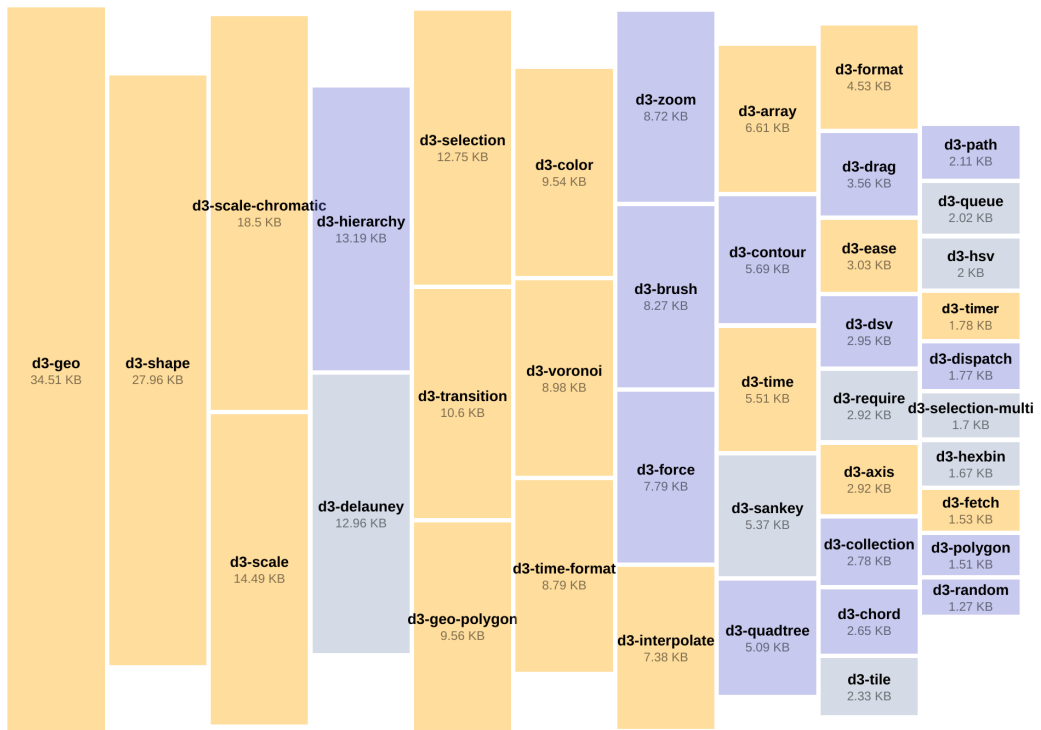
D3.js

Let's step back and look at all of the ground we've covered since we started this book. Here is a diagram of all of the d3.js modules, scaled by their minified size:



d3 modules in the core build

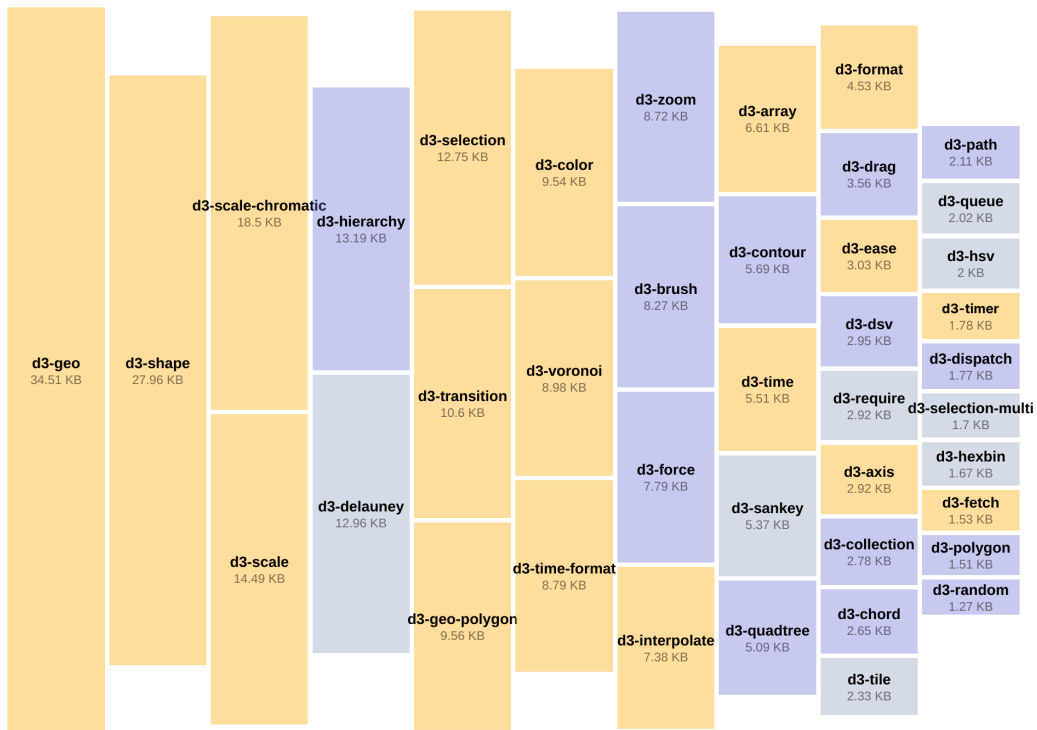
That's pretty overwhelming! But we've actually talked about almost half of them — here are the ones we've covered in yellow:



d3 modules that we've covered

What did we cover?

Let's look at the modules *we have* talked about:



d3 modules that we've covered

- **d3-geo**⁸⁹ helps with turning GeoJSON into 2D svg paths, and **d3-geo-polygon**⁹⁰ adds extra projections.
- **d3-shape**⁹¹ helps with creating shapes — like our timeline and pie chart.
- **d3-scale-chromatic**⁹² provides many color scales to use and **d3-color**⁹³ lets us manipulate colors to create our own color scales.
- **d3-scale**⁹⁴ helps us create scales to map from one domain to another. Be sure to explore the [docs](#)⁹⁵ to see what scale types we haven't covered, such as `d3.scaleLog()`, `d3.scaleThreshold()`, and `d3.scaleOrdinal()`, among others.

⁸⁹<https://github.com/d3/d3-geo>

⁹⁰<https://github.com/d3/d3-geo-polygon>

⁹¹<https://github.com/d3/d3-shape>

⁹²<https://github.com/d3/d3-scale-chromatic>

⁹³<https://github.com/d3/d3-color>

⁹⁴<https://github.com/d3/d3-scale>

⁹⁵<https://github.com/d3/d3-scale>

- **d3-selection**⁹⁶ helps with selecting and modifying elements on the webpage.
- **d3-transition**⁹⁷ helps with animating changes to our chart, and **d3-ease**⁹⁸ lets us change the timing of those animations.
- **d3-voronoi**⁹⁹ helps create voronoi polygons that cover our whole **bounds**, making it easy to find the closest data point from any location.
- **d3-format**¹⁰⁰ helps convert our data values into human-friendly strings, and **d3-time-format**¹⁰¹ helps convert date time objects into friendly strings.
- **d3-interpolate**¹⁰² helps us with interpolating in different color spaces, but it also works behind the scenes to interpolate other values for **d3-transition** (such as numbers, strings, and dates).
- **d3-array**¹⁰³ gives us utility methods for modifying and querying arrays, such as `d3.min()`, `d3.scan()`, and `d3.histogram()`.
- **d3-axis**¹⁰⁴ helps with creating axes — check out [the docs](#)¹⁰⁵ to see how to customize your ticks.
- and lastly, the first module we learned: **d3-fetch**¹⁰⁶, which helps to fetch and parse data files.

What did we not cover?

The d3 modules we haven't covered are more specialized — let's look at a few examples:

- **d3-hierarchy**¹⁰⁷ helps with creating hierarchical charts, such as treemaps and circle packing.

⁹⁶<https://github.com/d3/d3-selection>

⁹⁷<https://github.com/d3/d3-transition>

⁹⁸<https://github.com/d3/d3-ease>

⁹⁹<https://github.com/d3/d3-voronoi>

¹⁰⁰<https://github.com/d3/d3-format>

¹⁰¹<https://github.com/d3/d3-time-format>

¹⁰²<https://github.com/d3/d3-interpolate>

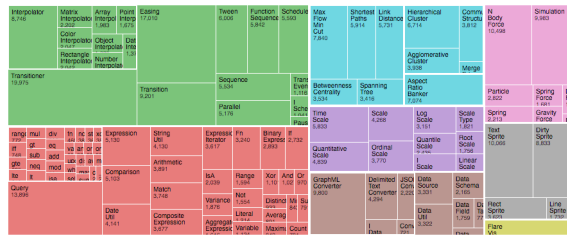
¹⁰³<https://github.com/d3/d3-array>

¹⁰⁴<https://github.com/d3/d3-axis>

¹⁰⁵<https://github.com/d3/d3-axis>

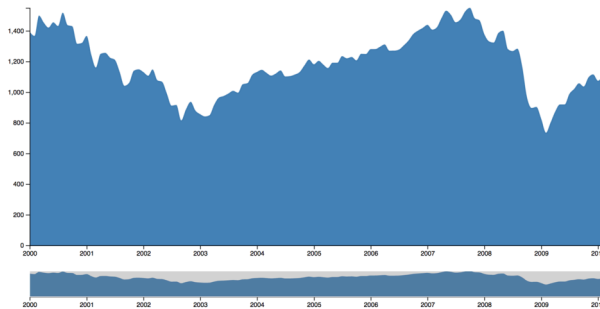
¹⁰⁶<https://github.com/d3/d3-fetch>

¹⁰⁷<https://github.com/d3/d3-hierarchy>



treemap example from <https://github.com/d3/d3-hierarchy>

- **d3-zoom**¹⁰⁸ and **d3-brush**¹⁰⁹ help with creating interactive charts, where the user can “zoom in” on a specific part.

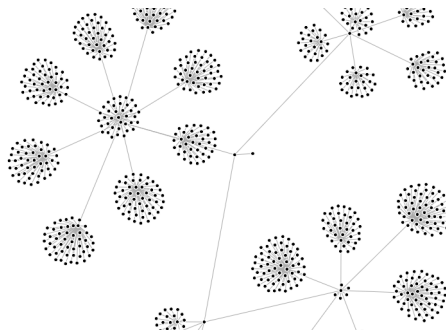
brushing example from <https://github.com/d3/d3-zoom>

- **d3-force**¹¹⁰ helps with placing nodes in network-like charts and beeswarm charts.

¹⁰⁸<https://github.com/d3/d3-zoom>

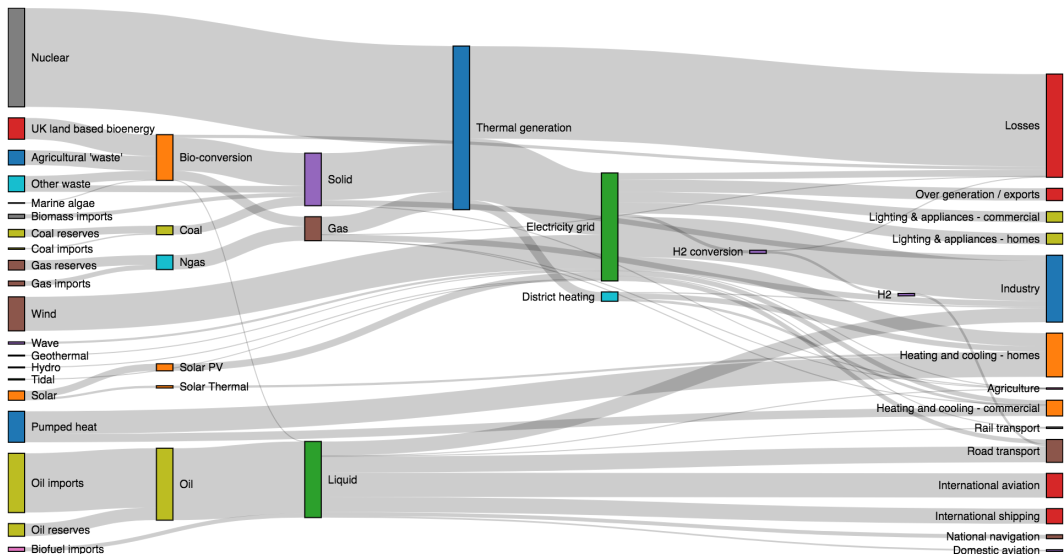
¹⁰⁹<https://github.com/d3/d3-brush>

¹¹⁰<https://github.com/d3/d3-force>



network diagram from <https://github.com/d3/d3-force>

- **d3-sankey**¹¹¹ specifically helps with creating Sankey diagrams, which can visualize flow and breakdowns.



sankey diagram from <https://github.com/d3/d3-sankey>

Now that you understand the basics of how to create a chart with d3.js and how the API generally works, I encourage you to explore these different modules. It's at least good to be aware of them — they might come in handy in the future.

¹¹¹<https://github.com/d3/d3-sankey>

Going forward

Great work learning all of that! As you can see, we covered *a lot* of concepts in this book — both how to implement it and also the theory behind how to design a good data visualization.

I'm sure you have tons of data you want to visualize — get out there and do it! Don't be overwhelmed by starting a chart by yourself — take it step by step (the chart steps are available in **Appendix B**), apply the fundamentals we've learned here, and refer back to the book when you need to pointer.

If you want ideas, here is a list of places to find inspiration:

Flowing Data¹¹² is a great, frequently-updated blog by Nathan Yau with original data visualizations and links to other complex data visualizations.

The Pudding¹¹³ is a digital publication that explores popular concepts with data visualizations. This is a great place to find visualizations that are interactive and easily digestible.

The New York Times Upshot¹¹⁴ often has interactive pieces that are linked to news events. Some of my favorite examples come from here, like their **You Draw It** article from 2015¹¹⁵ where *you guess what the data look like*.

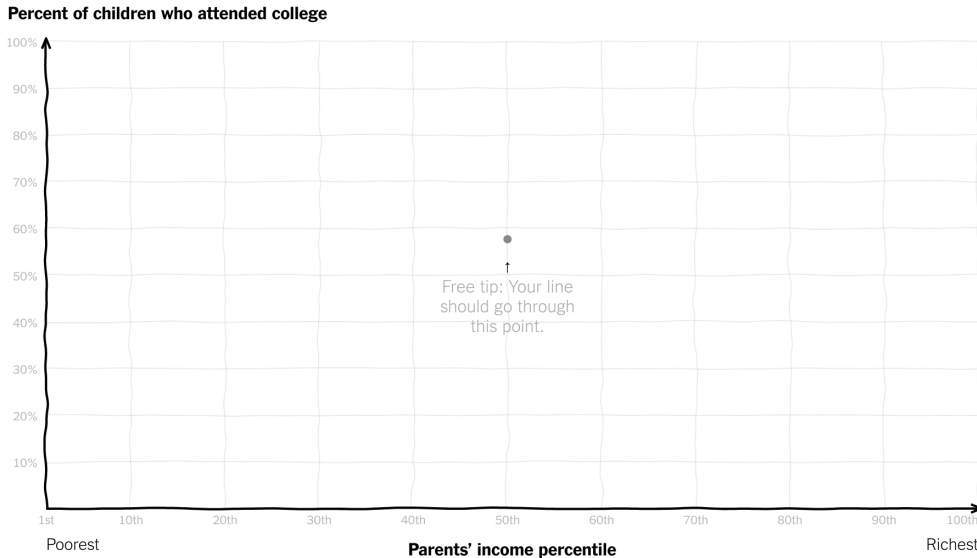
¹¹²<https://flowingdata.com/>

¹¹³<https://pudding.cool/>

¹¹⁴<https://www.nytimes.com/section/upshot>

¹¹⁵<https://www.nytimes.com/interactive/2015/05/28/upshot/you-draw-it-how-family-income-affects-childrens-college-chances.html?mtrref=wattenberger-book>

Draw your line on the chart below



You Draw It from (<https://www.nytimes.com/interactive/2015/05/28/upshot/you-draw-it-how-family-income-affects-childrens-college-chances.html>)

How was your experience?

I hope you found the book informative and fun! Did you find any part confusing or especially insightful? Send me an email at us@fullstack.io¹¹⁶ or tweet at me at [@wattenberger](https://twitter.com/wattenberger)¹¹⁷.

Send me your thoughts, feedback, and screenshots of your completed charts! If you've made anything you want to brag about with your new knowledge, I'd love to see it! Show it off and tag me in a tweet.

¹¹⁶<mailto:us@fullstack.io>

¹¹⁷<https://twitter.com/wattenberger>

Appendix

A. Generating our own weather data

Let's make sure that we have python 3 installed by running `python --version` in the terminal. If we get `command not found` or if the version is lower than 3.0.0, we'll need to upgrade first.

The Dark Sky API is a simple weather API that can tell what the weather was like at a specific time and place. The first 1,000 calls per day are free, which is perfect for our purposes. Go to darksky.net/dev/register and register for an account, then grab your apikey on the account screen.

Dark Sky API

DOCSFAQCONSOLEACCOUNT SETTINGSLOG OUT

Your Account

Your Secret Key

Your secret key grants you access to the Dark Sky API. You must supply the secret in all API requests.

XXXXXXXXXXXXXXXXXXXXXXXXXXXX

RESET SECRET KEY

Dark Sky apikey interface

Let's open up the `get_weather_data.py` file and update the `apikey` value, then grab our latitude and longitude from gps-coordinates.org¹¹⁸. Make sure to preserve the sign of the longitude (if it's negative, keep the minus sign).

Once we update those values we can run the script.

The script will use 365 of our 1,000 free daily api calls, so double-check the values before we run it.

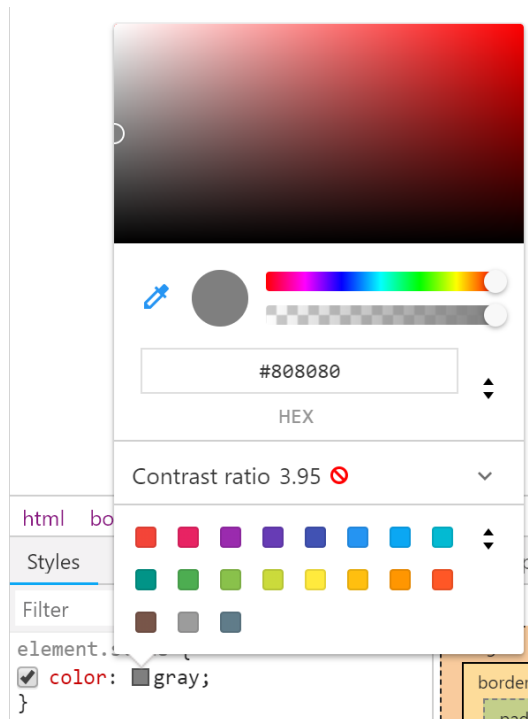
118 gps-coordinates.org

```
$ python get_weather_data.py
```

Now we should have a new `my_weather_data.json` file with the past 365 days of local weather. Let's take a closer look!

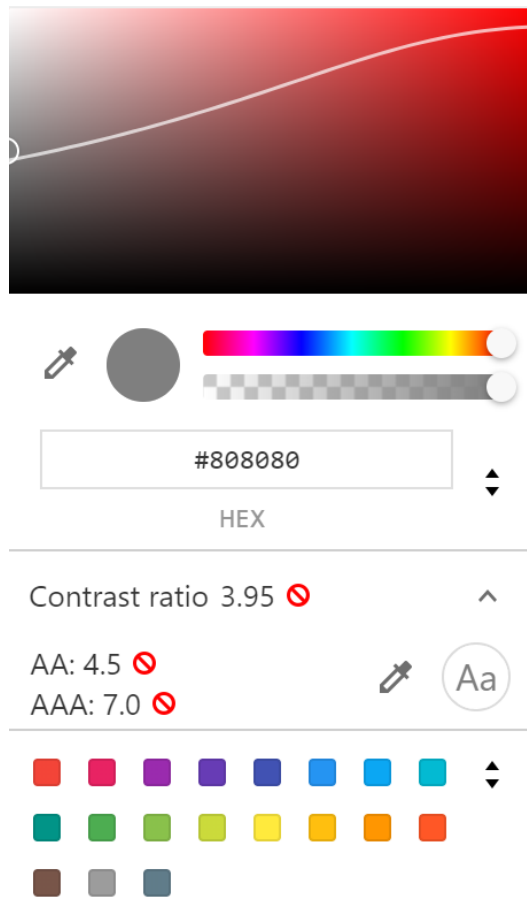
Chrome's Color Contrast Tool

If you're using Chrome, there is a great tool to check that your colors have enough contrast right in your dev tools. To check it out, you can load up one of our chart examples and inspect the wrapper element in the **Elements** panel. Give it a `color` property of `gray` and click the colored box to the left of `gray`.



color picker

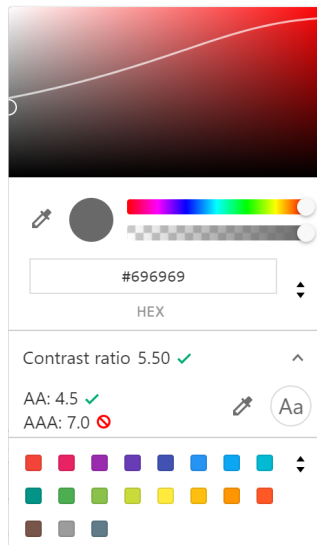
We can see the normal color picking controls, but also a **Contrast ratio** section with a warning symbol. Let's click the arrow to the right of the section to expand it.



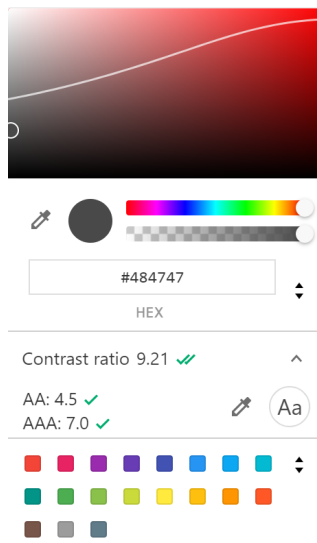
color picker expanded

Now we can see that the dev tools are making two checks: **AA** and **AAA**. These are checking for different contrast ratio levels – by **contrast ratio**, we’re talking about the difference in luminance between the text and its surroundings (higher means more contrast). **AA** is testing for at least a 4.5:1 contrast ratio (for people with 20/40 vision) and the **AAA** threshold is even higher at 7:1.

As we change the color by clicking around the color map at the top of the popup, we can see that values below the curved white line pass the **AA** threshold (one check mark) and values that are further below the white line pass the **AAA** threshold (two check marks).



color picker check



color picker checks

When making charts for your dashboard, it's a good idea to check your main colors every now and then to make sure they are visible to all users.

B. Chart-drawing checklist

These are the basic steps to follow to create a chart.

1. **Access data**

Look at the data structure and declare how to access the values we'll need

2. **Create chart dimensions**

Declare the physical (i.e. pixels) chart parameters

3. **Draw canvas**

Render the chart area and bounds element

4. **Create scales**

Create scales for every data-to-physical attribute in our chart

5. **Draw data**

Render your data elements

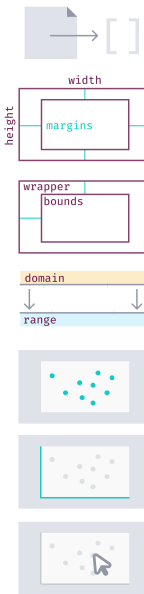
6. **Draw peripherals**

Render your axes, labels, and legends

7. **Set up interactions**

Initialize event listeners and create interaction behavior

Chart drawing checklist

☐ **Access data**

Look at the data structure and declare how to access the values we'll need

☐ **Create chart dimensions**

Declare the physical (i.e. pixels) chart parameters

☐ **Draw canvas**

Render the **wrapper** and **bounds** element

☐ **Create scales**

Create scales for every data-to-physical attribute in our chart

☐ **Draw data**

Render your data elements

☐ **Draw peripherals**

Render your axes, labels, legends, annotations, etc

☐ **Set up interactions**

Initialize event listeners and create interaction behavior

Checklist

C. SVG elements cheat sheet

SVG Elements

✓ can use CSS transitions

SVG is a language for drawing graphics declaratively. Here are the main elements that can be used to draw graphics on a web page.

svg

The Grand Poobah.

Use this to surround all other SVG elements or to create a new coordinate system.

rect

✓ x,y ✓ height ✓ width ✓

defs

The definitions element.

Use this to store elements to be used elsewhere. For example:

clipPath

Store in defs. reference: url(#id)

Used to clip other elements outside of its children elements' shape.

linearGradient, radialGradient

Store in defs. reference: url(#id)

Used to define a gradient, using:

stop

offset

✓ stop-color

✓ stop-opacity

line

x1,y1 x2,y2

circle

cx,cy ✓ r ✓

g

A container to group other SVG elements.

Similar to an HTML <div>.

polygon

points

path

d ✓

ellipse

cx,cy ✓ rx ✓ ry ✓

text

The only way to create text within SVG.

x,y

dx

dy

rotate

textLength

lengthAdjust

SVG elements cheat sheet

Changelog

Revision 15

04-01-2020

Chapter 13

Fix `useChartDimension` typo

Reset `Timeline.jsx` to use unfinished component imports

Thanks to David N. for reporting

Chapter 12

Switch to plain apostrophes to avoid text parsing issues

Thanks to David N. for reporting

Chapter 11

Fix `d3.timeMonth()` typo

Fix `UvOffset` typo

Thanks to David N. for reporting

Chapter 10

Fix “ot” typo

Thanks to David N. for reporting

Chapter 9

Fix typo in file name

Thanks to David N. for reporting

Chapter 8

Fix off-by-one issue

Fix typo in city name

Thanks to David N. for reporting

Chapter 7

Fix “cuveBasis” typo

Fix typo in file name

Thanks to David N. for reporting

Chapter 5

Fix “tricker” typo

Use `.html()` method to ensure degrees symbol is parsed correctly

Thanks to David N. for reporting

Chapter 4

Fix off-by-one issue when incrementing `selectedMetricIndex`

Thanks to David N. for reporting

Chapter 3

Fix `area-label` typo

Thanks to Robert C. for reporting

Revision 14

03-11-2020

Bump d3 version from v5.9.2 to v5.15.0

Chapter 7

Add new color scales

Chapter 5

Add note about `d3.leastIndex()` replacing `d3.scan()` in newer versions of d3-array

Thanks to Bruno E. for reporting

Revision 13

01-08-2020

Chapter 5

Fix typo (`rect` -> `rects`)

Thanks to Kurtis P. for reporting

Revision 12

12-09-2019

Chapters 8 - 12

Fix paths to d3 and data in code examples

Thanks to Yuqi for reporting

Revision 11

10-11-2019

Chapter 6

Fix line break in code example

Thanks to Marguerite R. for reporting

Chapter 8

Fix clipped sentence

Thanks to Marguerite R. for reporting

Chapter 11

Fix clipped code snippet

Thanks to Marguerite R. for reporting

Chapter 14

Add missing `npm install` command

Thanks to Marguerite R. for reporting

Revision 10

09-29-2019

Chapter 3

Update the number of bins in the example

Update the number of pixels above the chart for our `<line>`

Thanks to Ryo S. for reporting